NEXT ENGINEER · _SHEERAT_

Object Oriented Programming with Java

BCS403

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT, CSE-AIML, CSE-DS, AIDS | **Semester:** 4 | **Credits:** 4

L-T-P: 3-1-0 | **Scheme:** AKTU B.Tech 2nd year, 2023-24 onwards

UNIT 1 Java Fundamentals and OOP

UNIT 2 Exception Handling, I/O and Multithreading

UNIT 3 Java New Features

UNIT 4 Java Collections Framework

UNIT 5 Spring Framework and Spring Boot

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: comparison tables that examiners love ("differentiate between ...").
3. **Solved examples**: output-prediction questions and short programs, worked out.
4. **Code to remember**: the programs most often asked, written in compact form.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 and Unit 2. JVM architecture, OOP pillars, overloading vs overriding, abstract class vs interface, exception hierarchy, thread life cycle and synchronization.
- **Afternoon (2 to 3 hours):** Units 3 and 4. Lambdas, streams, records, sealed classes, switch expressions; then the collection hierarchy and Comparable vs Comparator.
- **Evening (2 hours):** Unit 5. IoC and DI, bean scopes and lifecycle, AOP terms, @SpringBootApplication and a REST controller with GET/POST/PUT/DELETE. Then the "Last-hour checklist".
- **In the exam:** for every concept, write a definition, a small code snippet and its output. A 7-mark answer with a correct program almost always scores well.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Java Fundamentals and OOP

✍️ 2-mark definitions

- **JVM (Java Virtual Machine):** the abstract machine that loads, verifies and executes bytecode. It makes Java platform independent ("write once, run anywhere").
- **JRE:** JVM plus the core class libraries needed to run Java programs.
- **JDK:** JRE plus development tools such as javac (compiler), java (launcher), javadoc and jar.
- **Bytecode:** the platform-neutral instruction set in .class files, produced by javac and run by the JVM.
- **JIT compiler:** part of the JVM that compiles frequently used bytecode into native machine code at run time for speed.
- **Class and object:** a class is a blueprint that defines fields and methods; an object is an instance of a class created with new.
- **Constructor:** a special method with the class name and no return type, called automatically when an object is created. If none is written, the compiler supplies a default no-argument constructor.
- **this keyword:** refers to the current object; used to separate fields from parameters and to call another constructor (this(...)).
- **static member:** belongs to the class, not to objects; shared by all objects and usable without creating one.
- **final:** a final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be extended.
- **Encapsulation:** binding data and methods together and hiding data with private fields accessed through getters and setters.
- **Abstraction:** showing only essential features and hiding details, using abstract classes and interfaces.
- **Polymorphism:** one name, many forms. Compile-time polymorphism is method overloading; run-time polymorphism is method overriding with dynamic method dispatch.
- **Package:** a namespace that groups related classes and interfaces, e.g. java.util. CLASSPATH tells the JVM where to find classes and packages.
- **JAR file:** a Java ARchive that bundles .class files and resources into one zip-format file, created with `jar cvf app.jar *.class`.

💡 Core concepts

Features of Java: simple, object-oriented, platform independent, secure (bytecode verifier, no explicit pointers), robust (strong typing, exception handling, garbage collection), multithreaded, portable, high performance (JIT), distributed and dynamic.

Compilation and execution: `Hello.java` → javac → `Hello.class` (bytecode) → class loader → bytecode verifier → interpreter / JIT → machine code.

JVM architecture: class loader subsystem (loading, linking, initialization) → runtime data areas (method area, heap, Java stacks, PC registers, native method stacks) → execution engine (interpreter, JIT, garbage collector) → native method interface.

Primitive data types

Type	Size	Default	Range / note
byte	1 byte	0	−128 to 127
short	2 bytes	0	−32,768 to 32,767
int	4 bytes	0	about $\pm 2.1 \times 10^9$
long	8 bytes	0L	about $\pm 9.2 \times 10^{18}$
float	4 bytes	0.0f	about 7 decimal digits
double	8 bytes	0.0d	about 15 decimal digits
char	2 bytes	'\u0000'	Unicode, 0 to 65,535
boolean	JVM dependent	false	true / false

Access modifiers

Modifier	Same class	Same package	Subclass (other package)	Everywhere
private	Yes	No	No	No
default (none)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Overloading vs overriding

Point	Overloading	Overriding
Where	same class (or subclass)	subclass
Signature	must differ in parameters	must be the same
Return type	can differ	same or covariant
Binding	compile time (static)	run time (dynamic)
Polymorphism	compile-time	run-time
static / private / final methods	can be overloaded	cannot be overridden

Abstract class vs interface

Point	Abstract class	Interface
Methods	abstract and concrete	abstract, default, static, private (Java 9+)
Variables	any kind	public static final only

Point	Abstract class	Interface
Constructor	allowed	not allowed
Inheritance	extends one class	a class can implement many interfaces
Keyword	abstract class, extends	interface, implements
Use	common base with shared code	a contract or capability

Types of inheritance in Java: single, multilevel and hierarchical with classes. Multiple and hybrid inheritance only through interfaces, because multiple class inheritance causes the diamond problem.

String, StringBuilder, StringBuffer

Point	String	StringBuilder	StringBuffer
Mutability	immutable	mutable	mutable
Thread safe	yes (immutable)	no	yes (synchronized)
Speed for many changes	slow	fastest	slower than StringBuilder

🧮 Solved examples

Example 1 (dynamic method dispatch).

```
class Shape { void draw() { System.out.println("Shape"); } }
class Circle extends Shape { void draw() { System.out.println("Circle"); } }
public class Test {
    public static void main(String[] a) {
        Shape s = new Circle();   // parent reference, child object
        s.draw();
    }
}
```

Output: `Circle`. The method called depends on the object type at run time, not on the reference type.

Example 2 (String comparison).

```
String a = "java", b = "java", c = new String("java");
System.out.println(a == b);       // true  (same literal in the string pool)
System.out.println(a == c);       // false (c is a new heap object)
System.out.println(a.equals(c));  // true  (same characters)
```

Example 3 (static vs instance).

```
class Counter {
    static int total = 0; int id;
    Counter() { total++; id = total; }
}
// new Counter(); new Counter(); new Counter();
// Counter.total is 3; each object has its own id: 1, 2, 3
```

Example 4 (constructor chaining). `new B()` prints `A` then `B`, because a subclass constructor first calls super() implicitly.

```
class A { A() { System.out.println("A"); } }
class B extends A { B() { System.out.println("B"); } }
```

⌨️ Code to remember

```
// Abstraction + interface + polymorphism in one program
interface Payable { double pay(); }

abstract class Employee implements Payable {
    protected String name;
    Employee(String name) { this.name = name; }
    abstract double pay();
    public String toString() { return name + " earns " + pay(); }
}

class FullTime extends Employee {
    private double salary;
    FullTime(String n, double s) { super(n); salary = s; }
    double pay() { return salary; }
}

class PartTime extends Employee {
    private int hours; private double rate;
    PartTime(String n, int h, double r) { super(n); hours = h; rate = r; }
    double pay() { return hours * rate; }
}

public class Main {
    public static void main(String[] args) {
        Employee[] staff = { new FullTime("Asha", 50000), new PartTime("Ravi", 40, 300) };
        for (Employee e : staff) System.out.println(e);
    }
}
// Asha earns 50000.0
// Ravi earns 12000.0
```

```
// Creating and using a package
// File: com/college/util/MathUtil.java
package com.college.util;
public class MathUtil { public static int square(int x) { return x * x; } }

// File: App.java
import com.college.util.MathUtil;
public class App { public static void main(String[] a) { System.out.println(MathUtil.square(5)); }
}
// Compile: javac -d . com/college/util/MathUtil.java App.java   Run: java App   Output: 25
```

⭐ Important questions

1. Explain the JVM architecture and how Java achieves platform independence.
2. Explain the OOP principles (encapsulation, inheritance, polymorphism, abstraction) with programs.
3. Differentiate between overloading and overriding, and between abstract class and interface, with code.
4. Explain constructors, constructor overloading, this and super with examples.
5. What are packages? Explain how to create, import and use a user-defined package; explain CLASSPATH and JAR files.

NEXT ENGINEER
sheerat

UNIT 2 Exception Handling, I/O and Multithreading

✍️ 2-mark definitions

- **Exception:** an abnormal event at run time that disturbs the normal flow of a program. It is an object of a subclass of Throwable.
- **Error:** a serious problem the program should not try to handle, e.g. OutOfMemoryError, StackOverflowError.
- **Checked exception:** checked by the compiler; must be caught or declared with throws, e.g. IOException, SQLException.
- **Unchecked exception:** a subclass of RuntimeException, not checked at compile time, e.g. ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException.
- **finally block:** always runs after try/catch, whether or not an exception occurred; used to release resources.
- **throw vs throws:** throw explicitly throws one exception object inside a method; throws in the method signature declares exceptions the method may pass on.
- **Stream:** a sequence of data. Byte streams (InputStream / OutputStream) handle 8-bit binary data; character streams (Reader / Writer) handle 16-bit Unicode text.
- **Thread:** a lightweight unit of execution inside a process that shares the process's memory.
- **Multithreading:** running two or more threads concurrently to make better use of the CPU.
- **Synchronization:** allowing only one thread at a time to access a shared resource, using the synchronized keyword (monitor lock).
- **Thread priority:** an int from 1 (MIN_PRIORITY) to 10 (MAX_PRIORITY); the default is 5 (NORM_PRIORITY).
- **Inter-thread communication:** threads cooperating with wait(), notify() and notifyAll(), which must be called inside synchronized code.
- **Deadlock:** two or more threads each waiting forever for a lock held by another.

💡 Core concepts

Exception hierarchy

- Throwable
- Error (OutOfMemoryError, StackOverflowError)
- Exception
  - Checked: IOException, FileNotFoundException, SQLException, ClassNotFoundException, InterruptedException
  - RuntimeException (unchecked): ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException, ClassCastException

Checked vs unchecked

Point	Checked	Unchecked
Checked by	compiler	not checked at compile time
Parent	Exception (not RuntimeException)	RuntimeException
Handling	must catch or declare	optional
Cause	outside conditions (file, network)	programming mistakes
Example	IOException	ArithmeticException

Rules of try/catch/finally. A try must have at least one catch or a finally. Catch more specific exceptions first (a catch for Exception before ArithmeticException is a compile error). finally runs even if the try returns; it is skipped only if System.exit() is called or the JVM crashes. Java 7 allows multi-catch: `catch (IOException | SQLException e)`.

Byte vs character streams

Point	Byte stream	Character stream
Unit	8-bit bytes	16-bit Unicode chars
Base classes	InputStream, OutputStream	Reader, Writer
File classes	FileInputStream, FileOutputStream	FileReader, FileWriter
Buffered	BufferedInputStream	BufferedReader (readLine())
Use	images, audio, binary	text files

Thread life cycle (Thread.State): NEW (created) → RUNNABLE (after start(), ready or running) → BLOCKED (waiting for a monitor lock) / WAITING (wait(), join()) / TIMED_WAITING (sleep(ms), wait(ms)) → back to RUNNABLE → TERMINATED (run() finished).

Creating threads

Extending Thread	Implementing Runnable
class T extends Thread, override run()	class R implements Runnable, implement run()
cannot extend another class	can still extend another class
start with new T().start()	start with new Thread(new R()).start()
each thread is a separate object	one Runnable can be shared by many threads

Calling run() directly runs the code in the current thread; only start() creates a new thread.

sleep() vs wait(): sleep is a static method of Thread and keeps the lock; wait is a method of Object, must be called in synchronized code and releases the lock until notify().

🧮 Solved examples

Example 1 (finally with return).

```
static int f() {
    try { return 1; }
    finally { System.out.println("finally"); }
}
// System.out.println(f());  prints "finally" then 1
```

Example 2 (order of handlers).

```
try {
    int[] a = new int[3];
    a[5] = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Divide by zero");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Bad index");
}
// Output: Divide by zero   (10 / 0 is evaluated before the array store)
```

Example 3 (custom exception).

```
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String msg) { super(msg); }
}
class Account {
    private double balance = 1000;
    void withdraw(double amt) throws InsufficientBalanceException {
        if (amt > balance) throw new InsufficientBalanceException("Balance too low");
        balance -= amt;
    }
}
// try { new Account().withdraw(5000); }
// catch (InsufficientBalanceException e) { System.out.println(e.getMessage()); }
// Output: Balance too low
```

Example 4 (file copy with character streams).

```
try (BufferedReader in = new BufferedReader(new FileReader("in.txt"));
     BufferedWriter out = new BufferedWriter(new FileWriter("out.txt"))) {
    String line;
    while ((line = in.readLine()) != null) { out.write(line); out.newLine(); }
} catch (IOException e) { System.out.println(e); }
```

⌨️ Code to remember

```
// Producer-consumer with wait() / notify()
class Buffer {
    private int item; private boolean full = false;

    synchronized void put(int x) throws InterruptedException {
        while (full) wait();          // wait until consumer takes the item
        item = x; full = true;
        System.out.println("Produced " + x);
        notify();
    }
    synchronized int take() throws InterruptedException {
        while (!full) wait();         // wait until producer puts an item
        full = false;
        System.out.println("Consumed " + item);
        notify();
        return item;
    }
}

public class PC {
    public static void main(String[] a) {
        Buffer b = new Buffer();
        new Thread(() -> { try { for (int i = 1; i <= 3; i++) b.put(i); } catch
(InterruptedException e) {} }).start();
        new Thread(() -> { try { for (int i = 1; i <= 3; i++) b.take(); } catch
(InterruptedException e) {} }).start();
    }
}
// Produced 1, Consumed 1, Produced 2, Consumed 2, Produced 3, Consumed 3
```

```
// Synchronized counter: without synchronized the final count is often below 2000
class Counter {
    private int count = 0;
    synchronized void inc() { count++; }
    int get() { return count; }
}
// Two threads each call inc() 1000 times; after t1.join() and t2.join(), get() is 2000.
```

⭐ Important questions

1. Explain the exception hierarchy. Differentiate checked and unchecked exceptions, and throw and throws. Write a program with a user-defined exception.
2. Explain try, catch, finally and nested try with examples.
3. Explain byte and character streams. Write a program to read from and write to a file.
4. Explain the thread life cycle. Compare the two ways of creating a thread.
5. What is synchronization? Explain inter-thread communication with the producer-consumer program.

UNIT 3 Java New Features

✍️ 2-mark definitions

- **Functional interface:** an interface with exactly one abstract method, optionally marked @FunctionalInterface, e.g. Runnable, Comparator, Predicate, Function, Consumer, Supplier.
- **Lambda expression (Java 8):** an anonymous function that implements a functional interface, written `(parameters) -> body`, e.g. `(a, b) -> a + b`.
- **Method reference:** a shorter form of a lambda that only calls an existing method, written with ::, e.g. `System.out::println`.
- **Stream API:** a java.util.stream pipeline for processing collections in a functional style: source → intermediate operations (lazy) → one terminal operation.
- **Default method:** an interface method with a body, marked default, so new methods can be added to interfaces without breaking implementing classes.
- **Static interface method:** a method with a body belonging to the interface, called as InterfaceName.method().
- **Base64:** java.util.Base64 provides encoders and decoders that turn binary data into text using 64 printable characters.
- **try-with-resources:** a try statement that declares resources (AutoCloseable) and closes them automatically at the end.
- **Annotation:** metadata attached to code, beginning with @, e.g. @Override, @Deprecated, @FunctionalInterface, @SuppressWarnings.
- **Module (Java 9):** a named group of packages with a module-info.java that states what it requires and exports.
- **var (Java 10):** local variable type inference; the compiler works out the type from the initializer, e.g. `var list = new ArrayList<String>();`.
- **Switch expression (Java 14):** a switch that returns a value, using arrow labels and yield.
- **Text block (Java 15):** a multi-line string literal written between `"""` delimiters.
- **Record (Java 16):** a compact, immutable data class; the compiler generates the constructor, accessors, equals, hashCode and toString.
- **Sealed class (Java 17):** a class that lists which classes may extend it with permits; subclasses must be final, sealed or non-sealed.

💡 Core concepts

Built-in functional interfaces (java.util.function)

Interface	Method	Takes → returns	Example
Predicate<T>	test	T → boolean	`n -> n > 0`
Function<T,R>	apply	T → R	`s -> s.length()`
Consumer<T>	accept	T → void	`x -> System.out.println(x)`

Interface	Method	Takes → returns	Example
Supplier<T>	get	() → T	`() -> Math.random()`
BinaryOperator<T>	apply	(T, T) → T	`(a, b) -> a * b`

Four kinds of method reference

Kind	Syntax	Lambda equivalent
Static method	`Math::abs`	`x -> Math.abs(x)`
Instance method of an object	`out::println`	`x -> out.println(x)`
Instance method of an arbitrary object	`String::length`	`s -> s.length()`
Constructor	`ArrayList::new`	`() -> new ArrayList<>()`

Stream operations

Intermediate (return a stream, lazy)	Terminal (produce a result)
filter, map, flatMap, distinct, sorted, limit, skip, peek	forEach, collect, reduce, count, min, max, anyMatch, allMatch, findFirst, toList

A stream does not store data, does not change its source, and can be used only once.

Feature timeline

Version	Feature
Java 7	try-with-resources, multi-catch, diamond operator
Java 8	lambdas, method references, Stream API, default and static interface methods, Base64, java.time
Java 9	module system, private interface methods
Java 10	var
Java 14	switch expressions
Java 15	text blocks
Java 16	records, pattern matching for instanceof
Java 17	sealed classes (LTS)

Annotations. Built-in: @Override, @Deprecated, @SuppressWarnings, @FunctionalInterface, @SafeVarargs. Meta-annotations: @Retention (SOURCE, CLASS, RUNTIME), @Target, @Documented, @Inherited, @Repeatable. Type annotations (Java 8) can be placed wherever a type is used; repeating annotations allow the same annotation more than once on a declaration.

🧮 Solved examples

Example 1 (stream pipeline).

```
List<Integer> nums = List.of(5, 12, 7, 20, 3, 18);
int sumOfSquaresOfEven = nums.stream()
        .filter(n -> n % 2 == 0)     // 12, 20, 18
        .map(n -> n * n)             // 144, 400, 324
        .reduce(0, Integer::sum);    // 868
System.out.println(sumOfSquaresOfEven);   // 868
```

Example 2 (collect and sort names).

```
List<String> names = List.of("ravi", "asha", "neha", "amit");
List<String> r = names.stream()
        .filter(s -> s.startsWith("a"))
        .map(String::toUpperCase)
        .sorted()
        .collect(Collectors.toList());
System.out.println(r);   // [AMIT, ASHA]
```

Example 3 (switch expression and text block).

```
int day = 6;
String type = switch (day) {
    case 1, 7 -> "Weekend";
    case 2, 3, 4, 5, 6 -> "Weekday";
    default -> { yield "Invalid"; }
};
System.out.println(type);   // Weekday

String html = """
        <p>Hello</p>
        """;
```

Example 4 (record and sealed class).

```
record Point(int x, int y) { }
Point p = new Point(2, 3);
System.out.println(p.x() + " " + p);   // 2 Point[x=2, y=3]

sealed interface Shape permits Circle, Square { }
final class Circle implements Shape { }
non-sealed class Square implements Shape { }
```

Example 5 (Base64).

```
String enc = Base64.getEncoder().encodeToString("Java".getBytes());
System.out.println(enc);                                          // SmF2YQ==
System.out.println(new String(Base64.getDecoder().decode(enc))); // Java
```

⌨️ Code to remember

```
// Interface with default and static methods, used with a lambda
@FunctionalInterface
interface Calculator {
    int compute(int a, int b);
    default void show(int a, int b) { System.out.println("Result: " + compute(a, b)); }
    static Calculator adder() { return (a, b) -> a + b; }
}
// Calculator mul = (a, b) -> a * b;   mul.show(4, 5);   prints Result: 20
// Calculator.adder().show(4, 5);                        prints Result: 9
```

```
// module-info.java for a module that uses java.sql and exposes one package
module com.college.app {
    requires java.sql;
    exports com.college.app.api;
}
```

⭐ Important questions

1. What is a functional interface? Explain lambda expressions and method references with examples.
2. Explain the Stream API with intermediate and terminal operations; write a program using filter, map and reduce.
3. Explain default and static methods in interfaces, try-with-resources, and Base64 encoding.
4. Explain records, sealed classes, switch expressions, text blocks and var with examples.
5. What are annotations? Explain built-in, type and repeating annotations, and the Java module system.

UNIT 4 Java Collections Framework

✍️ 2-mark definitions

- **Collection framework:** a set of interfaces (List, Set, Queue, Map), implementing classes and algorithms in java.util for storing and processing groups of objects.
- **Collection vs Collections:** Collection is the root interface; Collections is a utility class with static methods such as sort, reverse, max and shuffle.
- **Iterator:** an object that walks through a collection with hasNext(), next() and remove(). ListIterator can also move backward and add or set elements.
- **List:** an ordered collection that allows duplicates and index access (ArrayList, LinkedList, Vector, Stack).
- **Set:** a collection with no duplicates (HashSet, LinkedHashSet, TreeSet).
- **Queue:** a collection that usually orders elements first-in first-out (LinkedList, PriorityQueue, ArrayDeque). Deque allows insertion and removal at both ends.
- **Map:** stores key-value pairs with unique keys (HashMap, LinkedHashMap, TreeMap, Hashtable). Map is not a subtype of Collection.
- **Comparable:** java.lang interface with compareTo(T o) that defines the natural ordering inside the class itself.
- **Comparator:** java.util interface with compare(T a, T b) that defines a separate, custom ordering outside the class.
- **Properties:** a Hashtable subclass that stores String key-value pairs and can load and save them from .properties files.
- **Fail-fast iterator:** an iterator that throws ConcurrentModificationException if the collection is structurally changed during iteration (except through the iterator itself).

💡 Core concepts

Hierarchy

- Iterable → Collection
- List: ArrayList, LinkedList, Vector → Stack
- Set: HashSet → LinkedHashSet; SortedSet → NavigableSet → TreeSet
- Queue: PriorityQueue; Deque → ArrayDeque, LinkedList
- Map (separate): HashMap → LinkedHashMap; SortedMap → TreeMap; Hashtable → Properties

ArrayList vs LinkedList

Point	ArrayList	LinkedList
Structure	dynamic array	doubly linked list
get(i)	O(1)	O(n)
Insert / delete in middle	O(n), elements shift	O(1) once the node is found

Point	ArrayList	LinkedList
Memory	less	more (two links per node)
Implements	List	List and Deque
Best for	frequent reading	frequent insertion and deletion

Set implementations

Point	HashSet	LinkedHashSet	TreeSet
Order	no order	insertion order	sorted (natural or comparator)
Structure	hash table	hash table + linked list	red-black tree
null	one null allowed	one null allowed	not allowed
add / contains	O(1)	O(1)	O(log n)

Map implementations

Point	HashMap	LinkedHashMap	TreeMap	Hashtable
Order	none	insertion	sorted by key	none
null keys	one	one	none	none
Synchronized	no	no	no	yes (legacy)

Comparable vs Comparator

Point	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T o)	compare(T a, T b)
Where written	inside the class	separate class or lambda
Number of orderings	one natural order	many custom orders
Sorting call	Collections.sort(list)	Collections.sort(list, cmp) or list.sort(cmp)

Queue methods: offer / add insert, poll / remove remove the head, peek / element look at the head. The first of each pair returns null or false instead of throwing an exception. PriorityQueue removes the smallest element first (a min-heap) by default.

🧮 Solved examples

Example 1 (set ordering).

```
int[] data = {30, 10, 20, 10};
Set<Integer> h = new HashSet<>(), l = new LinkedHashSet<>(), t = new TreeSet<>();
for (int x : data) { h.add(x); l.add(x); t.add(x); }
System.out.println(l);   // [30, 10, 20]   insertion order, duplicate removed
System.out.println(t);   // [10, 20, 30]   sorted
// HashSet prints the same 3 elements in no guaranteed order
```

Example 2 (word frequency with HashMap).

```
String[] words = "to be or not to be".split(" ");
Map<String, Integer> freq = new TreeMap<>();
for (String w : words) freq.put(w, freq.getOrDefault(w, 0) + 1);
System.out.println(freq);   // {be=2, not=1, or=1, to=2}
```

Example 3 (PriorityQueue).

```
PriorityQueue<Integer> pq = new PriorityQueue<>(List.of(40, 10, 30, 20));
while (!pq.isEmpty()) System.out.print(pq.poll() + " ");   // 10 20 30 40
```

Example 4 (removing during iteration). Calling `list.remove(x)` inside a for-each loop over the same list throws ConcurrentModificationException. Use `Iterator.remove()` or `list.removeIf(x -> x < 0)` instead.

⌨️ Code to remember

```
// Sorting objects with Comparable (by roll) and Comparator (by marks, then name)
class Student implements Comparable<Student> {
    int roll; String name; double marks;
    Student(int r, String n, double m) { roll = r; name = n; marks = m; }
    public int compareTo(Student o) { return Integer.compare(roll, o.roll); }
    public String toString() { return roll + "-" + name + "-" + marks; }
}

public class SortDemo {
    public static void main(String[] a) {
        List<Student> list = new ArrayList<>(List.of(
                new Student(3, "Neha", 82), new Student(1, "Amit", 91), new Student(2, "Ravi",
82)));

        Collections.sort(list);                       // natural order by roll
        System.out.println(list);   // [1-Amit-91.0, 2-Ravi-82.0, 3-Neha-82.0]

        list.sort(Comparator.comparingDouble((Student s) -> s.marks).reversed()
                            .thenComparing(s -> s.name));
        System.out.println(list);   // [1-Amit-91.0, 3-Neha-82.0, 2-Ravi-82.0]
    }
}
```

```
// Properties: save and load configuration
Properties p = new Properties();
p.setProperty("db.user", "admin");
try (FileWriter w = new FileWriter("app.properties")) { p.store(w, "config"); }
Properties q = new Properties();
try (FileReader r = new FileReader("app.properties")) { q.load(r); }
System.out.println(q.getProperty("db.user"));   // admin
```

⭐ Important questions

1. Draw and explain the collection framework hierarchy.
2. Differentiate ArrayList and LinkedList, and HashSet, LinkedHashSet and TreeSet, with programs.
3. Explain the Map interface and its implementations; write a word-count program using HashMap.
4. Differentiate Comparable and Comparator; write a program to sort objects both ways.
5. Explain Iterator and ListIterator, the Queue and Deque interfaces, and the Properties class.

NEXT ENGINEER _sheerat_

UNIT 5 Spring Framework and Spring Boot

✍️ 2-mark definitions

- **Spring Framework:** a lightweight, open-source Java framework for building enterprise applications, based on IoC and DI.
- **Inversion of Control (IoC):** the framework, not the programmer, creates objects and manages their life cycle and dependencies.
- **IoC container:** the part of Spring that creates and wires beans. BeanFactory is the basic container; ApplicationContext adds events, internationalization and AOP support.
- **Dependency Injection (DI):** a form of IoC where the container supplies an object's dependencies through its constructor, a setter or a field.
- **Bean:** an object created, configured and managed by the Spring IoC container.
- **Bean scope:** how many instances of a bean the container creates and how long they live (singleton, prototype, request, session, application, websocket).
- **Autowiring:** automatic injection of matching beans, by type or by name, usually with @Autowired.
- **AOP (Aspect-Oriented Programming):** separating cross-cutting concerns such as logging, security and transactions from business logic.
- **Spring Boot:** a project built on Spring that offers auto-configuration, starter dependencies and an embedded server (Tomcat by default), so an application runs with almost no setup.
- **Starter:** a dependency descriptor that pulls in a group of libraries, e.g. spring-boot-starter-web.
- **@SpringBootApplication:** combines @Configuration, @EnableAutoConfiguration and @ComponentScan on the main class.
- **REST:** an architectural style where resources are identified by URLs and handled with HTTP methods GET, POST, PUT and DELETE, usually exchanging JSON.

💡 Core concepts

Types of dependency injection

Type	How	Notes
Constructor	dependencies passed to the constructor	recommended; supports final fields and required dependencies
Setter	dependencies set through setter methods	for optional dependencies
Field	@Autowired directly on the field	short, but harder to test

Bean scopes

Scope	Instances
singleton (default)	one per IoC container
prototype	a new one on every request to the container
request	one per HTTP request (web)
session	one per HTTP session (web)
application	one per ServletContext (web)
websocket	one per WebSocket session (web)

Autowiring modes (XML): no (default), byName, byType, constructor. With annotations, @Autowired works by type; @Qualifier("name") picks one bean when several match; @Primary marks the default choice.

Common annotations

Annotation	Purpose
@Component	generic Spring-managed bean
@Service, @Repository, @Controller	specialized components for business, data and web layers
@Configuration + @Bean	Java-based configuration; @Bean methods return beans
@Autowired, @Qualifier	inject dependencies
@Value("${key}")	inject a value from properties
@Scope("prototype")	set the bean scope
@PostConstruct, @PreDestroy	life-cycle callbacks

Bean life cycle: container starts → bean instantiated → dependencies injected → aware interfaces (BeanNameAware, ...) → BeanPostProcessor before-init → init callback (@PostConstruct, afterPropertiesSet() of InitializingBean, or init-method) → BeanPostProcessor after-init → bean ready to use → container closes → destroy callback (@PreDestroy, destroy() of DisposableBean, or destroy-method).

Configuration styles: XML-based (beans.xml with <bean> tags), annotation-based (@Component with component scanning) and Java-based (@Configuration classes with @Bean methods).

AOP terms

Term	Meaning
Aspect	a module holding a cross-cutting concern (@Aspect class)
Join point	a point in execution, e.g. a method call
Advice	the action taken at a join point: @Before, @After, @AfterReturning, @AfterThrowing, @Around
Pointcut	an expression that selects join points, e.g. `execution(* com.app.service.*.*(..))`
Weaving	linking aspects with target objects; Spring does it at run time with proxies

Spring vs Spring Boot

Point	Spring	Spring Boot
Configuration	mostly manual (XML or Java)	auto-configuration
Server	deploy WAR to an external server	embedded Tomcat, Jetty or Undertow
Dependencies	add each library	starters
Start-up	more boilerplate	run main() or `java -jar app.jar`

Spring Boot details.

- **Build systems:** Maven (pom.xml with spring-boot-starter-parent) or Gradle (build.gradle). Both support dependency management and the Spring Boot plugin to build an executable JAR.
- **Code structure:** keep the main class in the root package (e.g. com.example.app) so component scanning finds sub-packages: controller, service, repository, model.
- **Runners:** CommandLineRunner (run(String... args)) and ApplicationRunner (run(ApplicationArguments args)) run code once right after the application starts.
- **Logging:** Spring Boot uses SLF4J with Logback by default. Levels from most to least detailed: TRACE, DEBUG, INFO, WARN, ERROR. Set them in application.properties, e.g. `logging.level.com.example=DEBUG`.
- **REST annotations:** @RestController (= @Controller + @ResponseBody), @RequestMapping (base URL), @GetMapping, @PostMapping, @PutMapping, @DeleteMapping, @PathVariable (value from the URL path, /students/5), @RequestParam (value from the query string, ?name=x), @RequestBody (JSON body to object), ResponseEntity (body plus status code).

🧮 Solved examples

Example 1 (constructor injection with annotations).

```
interface MessageService { String send(String to); }

@Service
class EmailService implements MessageService {
    public String send(String to) { return "Email sent to " + to; }
}

@Component
class Notifier {
    private final MessageService service;
    Notifier(MessageService service) { this.service = service; }   // injected by Spring
    String notifyUser(String u) { return service.send(u); }
}
// notifier.notifyUser("asha")  returns "Email sent to asha"
```

With one constructor, Spring injects it automatically; @Autowired is optional.

Example 2 (singleton vs prototype). With singleton scope, `ctx.getBean(A.class) == ctx.getBean(A.class)` is true. With @Scope("prototype") on A, it is false because a new object is created each time.

Example 3 (logging aspect).

```
@Aspect
@Component
class LogAspect {
    @Before("execution(* com.example.app.service.*.*(..))")
    public void before(JoinPoint jp) {
        System.out.println("Calling " + jp.getSignature().getName());
    }
}
```

NEXT ENGINEER
sheerat

⌨️ Code to remember

```
// Spring Boot REST API: CRUD for students (in-memory)
@SpringBootApplication
public class App {
    public static void main(String[] args) { SpringApplication.run(App.class, args); }

    @Bean
    CommandLineRunner init() { return args -> System.out.println("Application started"); }
}

record Student(int id, String name) { }

@RestController
@RequestMapping("/students")
class StudentController {
    private final Map<Integer, Student> db = new HashMap<>();

    @GetMapping
    Collection<Student> all() { return db.values(); }

    @GetMapping("/{id}")
    ResponseEntity<Student> one(@PathVariable int id) {
        Student s = db.get(id);
        return s == null ? ResponseEntity.notFound().build() : ResponseEntity.ok(s);
    }

    @GetMapping("/search")
    List<Student> byName(@RequestParam String name) {
        return db.values().stream().filter(s -> s.name().equalsIgnoreCase(name)).toList();
    }

    @PostMapping
    ResponseEntity<Student> add(@RequestBody Student s) {
        db.put(s.id(), s);
        return ResponseEntity.status(HttpStatus.CREATED).body(s);
    }

    @PutMapping("/{id}")
    Student update(@PathVariable int id, @RequestBody Student s) {
        Student updated = new Student(id, s.name());
        db.put(id, updated);
        return updated;
    }

    @DeleteMapping("/{id}")
    ResponseEntity<Void> delete(@PathVariable int id) {
        db.remove(id);
        return ResponseEntity.noContent().build();
    }
}
// GET /students, GET /students/1, GET /students/search?name=asha,
// POST /students {"id":1,"name":"Asha"}, PUT /students/1, DELETE /students/1
```

```
# application.properties
server.port=8081
spring.application.name=student-api
logging.level.com.example=DEBUG
```

⭐ Important questions

1. Explain IoC and dependency injection; compare constructor, setter and field injection with code.
2. Explain bean scopes, autowiring, and the bean life cycle with callbacks.
3. Explain AOP concepts (aspect, advice types, join point, pointcut, weaving) with an example.
4. What is Spring Boot? Explain starters, auto-configuration, @SpringBootApplication, build systems, code structure, runners and logging.
5. Build a RESTful web service in Spring Boot showing @RestController, @RequestMapping, @PathVariable, @RequestParam, @RequestBody and GET, POST, PUT and DELETE.

Last-hour checklist

Read only this in the last hour before the paper.

- ☐ JDK ⊃ JRE ⊃ JVM; bytecode; JIT; JVM memory areas.
- ☐ Data type sizes; access modifier table; static and final.
- ☐ Overloading vs overriding; abstract class vs interface; dynamic dispatch output.
- ☐ String == vs equals; String vs StringBuilder vs StringBuffer.
- ☐ Exception hierarchy; checked vs unchecked; throw vs throws; finally with return.
- ☐ Byte vs character streams; BufferedReader.readLine().
- ☐ Thread states; Thread vs Runnable; start() vs run(); sleep() vs wait(); producer-consumer.
- ☐ Functional interfaces table; lambda; 4 method references; stream filter-map-reduce.
- ☐ Java 8 to 17 feature timeline; record; sealed permits; switch with yield; text block.
- ☐ Collection hierarchy; ArrayList vs LinkedList; HashSet vs TreeSet; HashMap vs TreeMap.
- ☐ Comparable (compareTo) vs Comparator (compare).
- ☐ IoC, DI types, bean scopes, life cycle; AOP terms; @SpringBootApplication = 3 annotations; REST controller.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Java: The Complete Reference (text book)	Herbert Schildt	McGraw Hill Education	WorldCat

#	Title	Author(s)	Publisher	Where to get it legally
2	Spring Boot in Action (text book)	Craig Walls	Manning Publications	WorldCat
3	Java Black Book	Steven Holzner	Dreamtech Press	WorldCat
4	Programming with Java	E. Balagurusamy	McGraw Hill	WorldCat
5	Java: A Beginner's Guide	Herbert Schildt	Oracle Press / McGraw Hill	WorldCat
6	Learning Spring Boot 2.0 (2nd ed.)	Greg L. Turnquist	Packt Publishing	WorldCat
7	Introduction to Java Spring Boot: Learning by Coding	AJ Henley Jr., Dave Wolf	Independently published	WorldCat

The official Java tutorials (dev.java/learn) and Spring guides (spring.io/guides) are free and legal companions for Units 3 and 5.

Official syllabus: AKTU B.Tech 2nd year CSE syllabus, 2023-24 scheme (PDF)

NEXT ENGINEER _sheerat_